

Python Operators

Complete Study Notes | Source: W3Schools

What Are Operators?

Operators are special symbols used to perform operations on variables and values. Python groups operators into **7 categories**: Arithmetic, Assignment, Comparison, Logical, Identity, Membership, and Bitwise.

Example: `print(10 + 5)` outputs 15

1. Arithmetic Operators

Used with numeric values to perform common mathematical operations.

Operator	Name	Example	Result / Note
+	Addition	<code>x + y</code>	Adds two values
-	Subtraction	<code>x - y</code>	Subtracts right from left
*	Multiplication	<code>x * y</code>	Multiplies two values
/	Division	<code>x / y</code>	Always returns a float
%	Modulus	<code>x % y</code>	Returns the remainder
**	Exponentiation	<code>x ** y</code>	Raises x to the power y
//	Floor Division	<code>x // y</code>	Rounds DOWN to nearest int

Tip: `//` rounds down even for negatives: `-7 // 2 = -4`.

2. Assignment Operators

Assign values to variables. Compound forms combine an operation with assignment.

Operator	Meaning	Equivalent
=	Assign	<code>x = 5</code>
+=	Add and assign	<code>x = x + 3</code>
-=	Subtract and assign	<code>x = x - 3</code>
*=	Multiply and assign	<code>x = x * 3</code>
/=	Divide and assign	<code>x = x / 3</code>
%=	Modulus and assign	<code>x = x % 3</code>
//=	Floor div and assign	<code>x = x // 3</code>
**=	Exponent and assign	<code>x = x ** 3</code>
&=	Bitwise AND assign	<code>x = x & 3</code>

=	Bitwise OR assign	x = x 3
^=	Bitwise XOR assign	x = x ^ 3
>>=	Right shift assign	x = x >> 3
<<=	Left shift assign	x = x << 3
:=	Walrus operator	print(n := len(a))

3. Comparison Operators

Compare two values and return **True** or **False**.

Operator	Name	Example	Returns True when...
==	Equal	x == y	x and y have the same value
!=	Not equal	x != y	x and y differ
>	Greater than	x > y	x is larger than y
<	Less than	x < y	x is smaller than y
>=	Greater than or equal	x >= y	x is larger or equal
<=	Less than or equal	x <= y	x is smaller or equal

Note: == compares; = assigns.

4. Logical Operators

Combine conditional statements; return a Boolean.

Operator	Description	Example	Returns True when...
and	Logical AND	x < 5 and x < 10	Both conditions are True
or	Logical OR	x < 5 or x < 4	At least one is True
not	Logical NOT	not(x < 5)	Condition is False (inverts it)

Short-circuit: and stops at first False; or stops at first True.

5. Identity Operators

Check whether two variables point to the **same object in memory**.

Operator	Description	Example	Returns True when...
is	Same object	x is y	x and y are identical objects
is not	Not same object	x is not y	x and y are different objects

Important: is tests identity, not equality. Two lists with equal content are NOT the same object.

6. Membership Operators

Test whether a value exists within a sequence (list, tuple, string, set, dict keys).

Operator	Description	Example	Returns True when...
<code>in</code>	Present in sequence	<code>x in y</code>	x exists in y
<code>not in</code>	Not present in sequence	<code>x not in y</code>	x does not exist in y

Works on: lists, tuples, strings, sets, range, dict (checks keys only).

7. Bitwise Operators

Operate on integers at the binary (bit) level.

Operator	Name	Example	Description
<code>&</code>	AND	<code>x & y</code>	Sets each bit to 1 if both bits are 1
<code> </code>	OR	<code>x y</code>	Sets each bit to 1 if either bit is 1
<code>^</code>	XOR	<code>x ^ y</code>	Sets each bit to 1 if only one bit is 1
<code>~</code>	NOT	<code>~x</code>	Inverts all the bits
<code><<</code>	Left shift	<code>x << 2</code>	Shifts bits left; multiplies by 2 per shift
<code>>></code>	Right shift	<code>x >> 2</code>	Shifts bits right; divides by 2 per shift

Example: $6 \& 3 \rightarrow 110 \& 011 = 010 = 2$

8. Operator Precedence (Highest to Lowest)

Python evaluates operators in the order below. Use parentheses to override.

Priority	Operator(s)	Description
1 (highest)	<code>()</code>	Parentheses
2	<code>**</code>	Exponentiation
3	<code>+x -x ~x</code>	Unary plus, minus, bitwise NOT
4	<code>* / // %</code>	Multiplication, Division, Modulus
5	<code>+ -</code>	Addition, Subtraction
6	<code><< >></code>	Bitwise shift operators
7	<code>&</code>	Bitwise AND
8	<code>^</code>	Bitwise XOR
9	<code> </code>	Bitwise OR
10	<code>== != > < >= <= is is not in not in</code>	Comparisons, Identity, Membership
11	<code>not</code>	Logical NOT
12	<code>and</code>	Logical AND

13 (lowest)	or	Logical OR
----------------	----	------------

Quick Code Examnles

```
# Arithmetic print(2 ** 8) # 256 print(17 // 3) # 5 print(17 % 3) # 2 # Comparison +
Logical x = 7 print(x > 5 and x < 10) # True print(not x == 7) # False # Membership
fruits = ['apple', 'banana'] print('apple' in fruits) # True # Identity a = [1, 2] b =
a print(b is a) # True (same object) c = [1, 2] print(c is a) # False (equal but
different objects) # Assignment shorthand x = 10 x += 5 # x is now 15 x **= 2 # x is now
225
```